IMC Prosperity 4 Round 2 Strategy: Algorithmic Trading, Bidding, and Manual Allocation

1. Round 1 Review and Round 2 Objective

Round 1 produced a total team PnL of 183,520 XIRECs, split between approximately 95,524 from algorithmic trading and 87,995 from manual trading, placing the team 1,139th overall and first in the manual component in that round. The Phase 1 advancement threshold is 200,000 net PnL after two rounds, so Round 2 must generate at least 16,480 net PnL to qualify, but any margin above 200,000 does not help Phase 2 directly. In Round 2, the algorithmic task remains trading INTARIAN_PEPPER_ROOT and ASH_COATED_OSMIUM under the same position limits of 80 units each, with a new choice of a one-time Market Access Fee (MAF) bid for 25% extra quote volume. Alongside this, the manual task is now an "Invest & Expand" allocation of a 50,000 XIREC budget across Research, Scale, and Speed, with payoff given by PnL = (Research × Scale × Speed) – Budget_Used.[1][2][3][4]

2. Current Algorithm Architecture (Round 1 Code)

The existing `Trader` class maintains separate logic for each product and uses persistent state stored via `traderData` (fast/slow moving averages and regime flags for the root, and an EMA anchor for osmium). For INTARIAN_PEPPER_ROOT, it computes a fast and slow exponential moving average of mid-price, defines slope = fast − slow and momentum = mid − last_mid, and sets a `hold_mode` flag when the trend is strongly positive or negative. When `hold_mode` is active, the root logic aggressively lifts offers up to fair + 2 until the long position hits the 80-unit limit; otherwise, it buys smaller chunks toward a target inventory of 40 units if slope ≥ 0 or 20 units if slope < 0 and then posts a passive bid near the fair value. For ASH_COATED_OSMIUM, the logic forms an EMA of the mid-price and a pseudo "fair value" anchor, then buys any ask at least one unit below fair, sells any bid at least one unit above fair, and finally sets symmetric passive bid/ask quotes around fair with a small inventory-based skew. Both legs therefore share three properties: a strong trend-following long bias in the root, a relatively tight mean-reversion band for osmium, and a modest but not aggressive use of passive orders.[^5]

3. Empirical Structure in Round 2 Price Data

Round 2 price files for day −1, 0, and 1 provide full best-three-level order book snapshots and profit columns for both products over many thousands of timestamps. Intarian Pepper Root prices in these

files show slow but persistent intraday drifts over hundreds of ticks, with spreads typically one or two units and liquid depth close to the top of book, making trend-following with moderate holding periods profitable if entries are not too aggressive. Ash-Coated Osmium mid-prices are more stable with frequent short-lived deviations of roughly one to three units around a slowly drifting level, and spreads shift between tight (two units) and wider in more volatile segments. The profit column confirms that systematically buying when quotes are a couple of ticks cheap relative to mid and selling when they are a couple of ticks rich can generate PnL, but overly aggressive crossing can erode returns, especially once any access fee is subtracted.[6][4][^7]

4. Simplifying and Strengthening the INTARIAN_PEPPER_ROOT Logic

The current root logic mixes regime detection, aggressive active buying, and inventory management in a single block, which increases the chance of fragile behaviour when Round 2 distributions shift. A simpler and more robust structure is:[^5]

- Regime detection: continue using fast/slow EMA and momentum but use only one composite signal such as `trend_score = 0.5 * slope + 0.5 * momentum` and three regimes (strong up, neutral, strong down).
- Inventory bands: instead of a "hard" hold-mode store-to-80, define a floating target inventory, e.g. `target_pos = 60` in strong up, `30` in neutral, and `10` in strong down, capped by the 80 limit.
- Entry discipline: restrict market-taking to asks at or below `min(best_ask, fair + 1)` and size each fill to a small chunk (e.g. up to 10 units) until current position approaches target.

This structure keeps the core Round 1 edge—using the trend to justify a long bias—but avoids situations where the bot rapidly jumps from a small position to the full 80 at poor prices, which hurt PnL in some Round 1 sequences. Because Round 2 test harness uses roughly 80% of all generated quotes with some randomization, keeping the logic simple and monotone in the trend signal reduces overfitting to noise in the example data. Finally, unwinds should be tied symmetrically to clear trend reversals and rich bids (e.g. sell on bids ≥ fair + 2 when trend_score is negative), rather than only slope thresholds, which helps the bot exit stale longs gracefully.[1][8][^5]

5. Refining ASH_COATED_OSMIUM Market-Making

The osmium strategy already behaves like a mild market-maker around its EMA-based fair value but uses a fixed offset of one tick and static order size up to 20 units. Given the Round 2 price behaviour, the process can be strengthened by:[^5]

- Re-anchoring fair: treat fair as a convex combination of EMA and mid, e.g. fair = 0.7 × EMA + 0.3 × mid, to react a bit faster when the market trends without abandoning mean-reversion.

- Dynamic spread: set the half-spread to
  - 1 when observed spread ≤ 3,
  - 2 when 3 < spread ≤ 6,
  - 3 (current) when spread > 6,
    so that quotes stay reasonably tight in normal conditions but do not sit at obvious arbitrage points in wide markets.[^6]
- Position-aware size: instead of fixed 20-lot orders, scale the order size with remaining inventory room (e.g. size = min(10, limit − |pos|)), which reduces the chance of over-filling near the limit.

Additionally, inventory skew should be made stronger and linear in position—for example, skew = `k * pos` with k around 0.1—so that the quote naturally leans away from existing inventory and pushes the bot to mean-revert its book. Combined with a stricter mispricing condition (only lift asks ≤ fair − 1.5 and hit bids ≥ fair + 1.5), this favours clearer edge trades and fewer marginal fills that contribute volume but little PnL.[6][5]

6. Implementation Check-List for Updating `trader.py`

When editing the Round 2 `Trader` implementation, a clean workflow is:

1. **Separate functions per product.** Create helper functions like `trade_root(state, depth, data)` and `trade_osmium(...)` to keep logic readable and easily tunable.[^5]
2. **Parameterize constants.** Define all tuning parameters at the top (fast and slow alphas, target positions, fair offsets, skew coefficient), so that they can be adjusted in small increments between submissions without rewriting logic.[^5]
3. **Add a `bid()` method.** Introduce the Market Access Fee bid inside the Trader class as `def bid(self): return your_number`, ensuring that it does not depend on runtime state and is easy to experiment with.[^1]
4. **Log key metrics offline.** When locally replaying the Round 2 example data, log time series of position, PnL, and fill counts for each product to confirm inventory never breaches ±80 and that turnover is spread through the day rather than clustered in a few bursts.[6][7]
5. **Protect against missing data.** Preserve the early-exit when either side of the book is empty, and include safe defaults for EMA and state fields so that restarts do not create erratic behaviour.[^5]

This code-level discipline reduces complexity without losing the main components that already worked in Round 1 and sets up an iterative tuning process where only parameters, not structure, are changed between submissions.[^8]

7. Market Access Fee (MAF) Strategy

The MAF auction is a one-shot, first-price auction in which only the top 50% of bids receive the additional 25% of generated quotes, and winners pay exactly their bid, which is subtracted from their final Round 2 PnL. During local testing, the bid is ignored, and all teams see only 80% of the generated quotes; the actual comparison of bids and addition of extra quotes occurs once at the final simulation, making the bid essentially a blind auction parameter. Because only being above the median matters—there is no extra reward for the highest bid—the rational objective is to choose the smallest bid that still leaves a high probability of landing above the median; bidding "too brave" simply raises the PnL hurdle you must clear to stay above 200k.[9][10][^1]

Using the team's Round 1 performance as a prior, the marginal gain from 25% extra quotes can be approximated as some percentage of the Round 2 base algorithmic PnL; if extra access boosts expected Round 2 PnL by, say, 10–20%, then paying more than 10–20% of that incremental expectation is structurally negative EV. The teammate strategy memo recommends treating the bid as an insurance premium and targeting a "low-to-mid tier" safety bid in the range 2,000–5,000 XIRECs, leveraging odd numbers (e.g. 2,437 instead of 2,400) to jump above round-number bids at essentially no extra cost. Given that many teams are inactive or low-Pnl and that Round 2 is constrained by the 200k cap, this range is a reasonable starting point; it can be nudged up or down slightly depending on back-tested estimates of how much additional PnL the 25% volume yields on your own strategy.[11][9]

8. Manual Challenge: Optimal Research–Scale–Speed Allocation

The manual challenge offers a 50,000 XIREC budget to allocate across Research, Scale, and Speed, where Research grows logarithmically from 0 to 200,000 as investment goes from 0 to 100, Scale grows linearly from 0 to 7, and Speed is a rank-based multiplier between 0.1 and 0.9 depending on how your Speed percentage compares to other teams. The objective function is[^2]

$$PnL = (Research(x) \times Scale(y) \times Speed(z)) - 50000,$$

[^12]

with x, y, z in and x + y + z ≤ 100, tied via the 50,000 budget. Because Research is logarithmic, its marginal benefit declines at high allocations, while Scale and Speed keep adding linearly or via rank; therefore, putting all 100 into one pillar is sub-optimal compared to a balanced allocation that keeps each factor in a productive region.[^2]

Under symmetric assumptions about how other players allocate, a robust heuristic is to favour Research slightly, keep Scale in a strong but not extreme range, and devote a solid chunk to Speed to avoid a bottom-quartile rank. For example:

- Research: 40–45
- Scale: 30–35
- Speed: 20–25

This keeps Research well into the concave region where it multiplies the output materially, while giving enough Scale and Speed that the product of the three terms is large before deducting the fixed 50,000 cost. Since Speed is rank-based, if competition appears extremely aggressive (e.g. public advice pushing most teams to Speed 40–50), it can be rational to underweight Speed slightly and rely more on Research×Scale, but given uncertainty, a middle-of-the-road Speed share with a non-round pattern (e.g. 43/32/25) reduces the chance of tying exactly with many teams. Remember that allocations can be resubmitted until the end of the round, so if updated official hints shift the meta, the team can adjust proportions accordingly.[^2]

9. Practical Process for Strategy Iteration

Within the limited submission window, the most effective way to improve both algorithmic and manual decisions is to follow a repeatable cycle of analyse → adjust parameters → re-test.

For the algorithmic component:

- Run offline replays of Round 2 price data for day –1, 0, and 1, capturing per-product PnL, inventory trajectories, and order counts for each strategy tweak.[6][7]
- Change only one parameter family at a time (e.g. root target positions, osmium mispricing thresholds) so that PnL differences can be meaningfully attributed to specific changes.[^5]
- Track not only total PnL but also maximum drawdown and the worst single-day loss in each product; strategies with smoother PnL and lower risk of large losses are more robust when the unseen final simulation differs from the sample data.[^4]

For manual and bidding decisions:

- Use simple spreadsheet models or code to evaluate Research × Scale × Speed trade-offs under multiple assumed Speed ranks, to see how sensitive final PnL is to changes in each pillar.[^2]
- Treat the MAF bid as a separate variable and examine scenarios such as “no access,” “access plus 5k bid,” and “access plus 10k bid” using realistic estimates of incremental algorithmic PnL from extra volume.[1][11]

- Lock in final choices a little before the deadline to avoid last-minute errors, but keep an eye on any new hints or wiki updates that might materially shift optimal bid or allocation levels.[^13]

10. Beginner-Friendly Role Split and Learning Path

Given that the user identifies as a beginner, Round 2 preparation should allocate tasks according to team strengths while offering a structured path to learn trading fundamentals. One teammate with stronger coding experience should maintain and refactor `trader.py`, while the beginner focuses on understanding the high-level logic (trend-following vs mean-reversion), checking parameters, and running backtests on the price CSVs. The beginner can also take ownership of the manual challenge by building a small tool (spreadsheet or short script) that computes Research×Scale×Speed for different allocation triples, deepening intuition about diminishing returns and risk–reward trade-offs.[2][6][^5]

Conceptually, the beginner should prioritise learning:

- Order book basics: bid/ask, spread, depth, and how limit vs market orders behave under the provided simulator.[^1]
- Trend and mean-reversion signals: how moving averages, slope, and momentum are computed and what they imply for desired position.[^5]
- Simple risk control: position limits, inventory mean-reversion, and why gradual sizing is safer than jumping to the full limit.

By Round 2's end, this structured involvement both improves team performance and provides a solid foundation for later phases, where more complex products and multi-leg strategies may appear.[^1]

References

1. Screenshot-2026-04-18-at-08.04.27.jpg
2. Screenshot-2026-04-18-at-08.04.40.jpg
3. Screenshot-2026-04-18-at-08.03.42.jpg
4. prices_round_2_day_1.csv
5. 271680.py - from datamodel import OrderDepth, TradingState, Order
   from typing import List, Dict
   import jsonpickl...

6. prices_round_2_day_0.csv - daytimestampproductbidprice1bidvolume1bidprice2bidvolume2bidprice3bidvolume3askprice1askvolume1askpr...
7. prices_round_2_day_-1.csv
8. 271680.json - COATEDOSMIUM99891499862710007279998.03821.43751253700INTARIANPEPPERROOT1324712132432313264231325 5.51...
9. Screenshot-2026-04-18-at-07.58.25.jpg
10. Screenshot-2026-04-18-at-08.05.36.jpg
11. Bidding-Strategy-for-Auction-Survival.docx - Here is the strategic summary of our bidding plan for Round 2, formatted for you to share with your ...
12. trades_round_2_day_-1.csv
13. Screenshot-2026-04-18-at-08.05.14.jpg